



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验设计报告

开课学期: 2024 年秋季
课程名称: 操作系统
实验名称: 锁机制的应用
实验性质: 课内实验
实验时间: 10.24 地点: T2608
学生班级: 5
学生学号: 220110515
学生姓名: 金正达
评阅教师: _____
报告成绩: _____

实验与创新实践教育中心印制

2024 年 9 月

一、 回答问题

1、 内存分配器

a. 什么是内存分配器？它的作用是？

内存分配器是操作系统的一个组件，负责管理程序运行时的内存分配和释放。在程序请求内存时，找到并分配合适大小的内存块；当内存不再需要时，负责回收这部分内存。

b. 内存分配器的数据结构是什么？它有哪些操作（函数），分别完成了什么功能？

数据结构是由空闲物理页组成的链表。

kinit: 初始化链表和锁

kfree: 释放对应的内存块

kalloc: 分配空闲的内存块

c. 为什么指导书提及的优化方法可以提升性能？

可以降低多个 CPU 对空闲内存块的竞争，原来的设计只有一个全局锁，一个 CPU 申请内存时，其他 CPU 需要等待；改进之后，每个 CPU 有自己对应的内存锁，不必竞争锁导致等待。

2、 磁盘缓存

a. 什么是磁盘缓存？它的作用是？

磁盘缓存是磁盘与文件系统交互的中间层，是一种用于在内存中临时保存从磁盘读取的数据结构。

作用是：减少磁盘的直接访问次数，提高系统性能；通过缓存在内存中的数据，加快数据的读取与写入，提升文件系统的整体效率。

b. **buf** 结构体为什么有 **prev** 和 **next** 两个成员，而不是只保留其中一个？请从这样做的优点分析（提示：结合通过这两种指针遍历链表的具体场景进行思考）。

双向链表可以进行高效的插入和删除操作，删除时不需要从头遍历获取被删除节点的前驱节点，有利于提升在访问某个缓存块后将其移至链表头部的操作的性能；在删除最后一个节点时，可以直接通过头部节点的前驱找到最后一个节点进行删除。高效的插入与删除操作有利于 LRU 算法的简洁实现。

c. 为什么哈希表可以提升磁盘缓存的性能？可以使用内存分配器的优化方法优化磁盘缓存吗？请说明原因。

每个哈希桶占有一个自旋锁，相比于整个缓存块占用一个自旋锁，可以大大降低多个进程对缓存锁资源的竞争。同时使用哈希表进行索引也提高了在缓存池中寻找对应缓存的速度。

可以使用内存分配器优化缓存，通过提高缓存块的分配和回收效率、减少内存碎片、引入内存池管理，可以显著降低管理缓存块时的内存分配开销，进而提高整体系统性能。

二、实验详细设计

注意不要照搬实验指导书上的内容，请根据你自己的设计方案来填写

实验一：

1. 使多个 CPU 分别享有一个 freelist:

```
struct kmem kmems[NCPU];
```

2. kinit 中初始化所有 freelist 的锁:

```
for(int i = 0; i < NPCU; i++) {
    initlock(&kmems[i].lock, "kmem");
}
```

3. kfree 中针对对应 cpu 的 freelist 进行释放:

```
push_off();
int cid = cpuid();
acquire(&kmems[cid].lock);
r->next = kmems[cid].freelist;
kmems[cid].freelist = r;
release(&kmems[cid].lock);
pop_off();
```

4. kalloc 中，先判断当前 cpu 的 freelist 中是否有空余:

```
79 push_off();
80 int cid = cpuid();
81 acquire(&kmems[cid].lock);
82 r = kmems[cid].freelist;
83 if(r) {
84     kmems[cid].freelist = r->next;
85 } else {
```

如果没有，就从其他 CPU 的 freelist 中获取空余:

```
85 } else {
86     for (int i = 0; i < NPCU; i++) {
87         if (i == cid) continue;
88         acquire(&kmems[i].lock);
89         r = kmems[i].freelist;
90         if (r) {
91             kmems[i].freelist = r->next;
92             release(&kmems[i].lock);
93             break;
94         }
95         release(&kmems[i].lock);
96     }
97 }
98 release(&kmems[cid].lock);
99 pop_off();
```

实验二：

1. 修改缓存池的结构，增加哈希桶和每个哈希桶对应的锁:

```

struct {
    struct spinlock locks[NBUCKETS];
    struct buf buf[NBUF];

    // Linked list of all buffers, through prev/next.
    // Sorted by how recently the buffer was used.
    // head.next is most recent, head.prev is least.
    struct buf head;
    struct buf buckets[NBUCKETS];
} bcache;

```

2. 修改 binit, 初始化每个哈希桶的锁, 将缓存块加入各个哈希桶中:

```

for (int i = 0; i < NBUCKETS; i++) {
    initlock(&bcache.locks[i], "bcache");
    bcache.buckets[i].prev = &bcache.buckets[i];
    bcache.buckets[i].next = &bcache.buckets[i];
    for (b = bcache.buf + i * NBUF/NBUCKETS; b < bcache.buf + (i + 1) * NBUF/NBUCKETS; b++) {
        b->next = bcache.buckets[i].next;
        b->prev = &bcache.buckets[i];
        initsleeplock(&b->lock, "buffer");
        bcache.buckets[i].next->prev = b;
        bcache.buckets[i].next = b;
    }
}

```

3. 修改 bget, 首先获取对应 hash 值的桶的锁:

```
int hash = blockno % NBUCKETS;
```

```
acquire(&bcache.locks[hash]);
```

获取对应 hash 值的桶的锁后, 在该桶中查找磁盘内容是否已经缓存:

```

for(b = bcache.buckets[hash].next; b != &bcache.buckets[hash]; b = b->next){
    if(b->dev == dev && b->blockno == blockno){
        b->refcnt++;
        release(&bcache.locks[hash]);
        acquiresleep(&b->lock);
        return b;
    }
}

```

如果未缓存, 在该桶中查找 LRU 的 buf 进行替换:

```

for(b = bcache.buckets[hash].prev; b != &bcache.buckets[hash]; b = b->prev){
    if(b->refcnt == 0) {
        b->dev = dev;
        b->blockno = blockno;
        b->valid = 0;
        b->refcnt = 1;
        release(&bcache.locks[hash]);
        acquiresleep(&b->lock);
        return b;
    }
}

```

如果未找到可替换的 buf, 先释放当前桶的锁:

```
release(&bcache.locks[hash]);
```

从其他的哈希桶寻找一个可以被替换的 buf, 在寻找过程中, 先获得对应桶的锁, 将 buf 从对应桶的双向链表中移除后, 释放对应桶的锁, 再获得原来对应 hash 值的桶的锁, 将获得的 buf 插入后释放锁:

```

for (int i = 0; i < NBUCKETS; i++) {
    if (i == hash) continue;
    acquire(&bcache.locks[i]);
    for(b = bcache.buckets[i].prev; b != &bcache.buckets[i]; b = b->prev) {
        if(b->refcnt == 0) {
            b->dev = dev;
            b->blockno = blockno;
            b->valid = 0;
            b->refcnt = 1;
            b->next->prev = b->prev;
            b->prev->next = b->next;
            release(&bcache.locks[i]);
            acquire(&bcache.locks[hash]);
            b->prev = &bcache.buckets[hash];
            b->next = bcache.buckets[hash].next;
            bcache.buckets[hash].next->prev = b;
            bcache.buckets[hash].next = b;
            release(&bcache.locks[hash]);
            acquiresleep(&b->lock);
            return b;
        }
    }
    release(&bcache.locks[i]);
}
}

```

4. 修改 `brelse`，使其针对具体的 `hash` 桶进行：

```

int hash = b->blockno % NBUCKETS;
acquire(&bcache.locks[hash]);
b->refcnt--;
if (b->refcnt == 0) {
    // no one is waiting for it.
    b->next->prev = b->prev;
    b->prev->next = b->next;
    b->next = bcache.buckets[hash].next;
    b->prev = &bcache.buckets[hash];
    bcache.buckets[hash].next->prev = b;
    bcache.buckets[hash].next = b;
}
release(&bcache.locks[hash]);

```

5. 同样修改 `bpin` 和 `bunpin`:

```

void
bpin(struct buf *b) {
    int hash = b->blockno % NBUCKETS;
    acquire(&bcache.locks[hash]);
    b->refcnt++;
    release(&bcache.locks[hash]);
}

void
bunpin(struct buf *b) {
    int hash = b->blockno % NBUCKETS;
    acquire(&bcache.locks[hash]);
    b->refcnt--;
    release(&bcache.locks[hash]);
}

```

三、实验结果截图

请给出 `kalloctest`、`bcachetest`、`usertests` 及 `make grade` 的测试结果。

Kalloctest:

```
$ kalloctest
start test1
test1 results:
--- lock kmem/bcache stats
lock: kmem: #fetch-and-add 0 #acquire() 59764
lock: kmem: #fetch-and-add 0 #acquire() 185998
lock: kmem: #fetch-and-add 0 #acquire() 187285
lock: bcache: #fetch-and-add 0 #acquire() 2
lock: bcache: #fetch-and-add 0 #acquire() 4
lock: bcache: #fetch-and-add 0 #acquire() 10
lock: bcache: #fetch-and-add 0 #acquire() 12
lock: bcache: #fetch-and-add 0 #acquire() 20
lock: bcache: #fetch-and-add 0 #acquire() 14
lock: bcache: #fetch-and-add 0 #acquire() 282
lock: bcache: #fetch-and-add 0 #acquire() 8
lock: bcache: #fetch-and-add 0 #acquire() 2
lock: bcache: #fetch-and-add 0 #acquire() 2
--- top 5 contended locks:
lock: proc: #fetch-and-add 36366 #acquire() 129573
lock: uart: #fetch-and-add 12889 #acquire() 117
lock: proc: #fetch-and-add 5745 #acquire() 129610
lock: pr: #fetch-and-add 4411 #acquire() 5
lock: proc: #fetch-and-add 4026 #acquire() 129579
tot= 0
test1 OK
start test2
total free number of pages: 32492 (out of 32768)
.....
test2 OK
```

Bcachetest:

```
$ bcachetest
start test0
test0 results:
--- lock kmem/bcache stats
lock: kmem: #fetch-and-add 0 #acquire() 580596
lock: kmem: #fetch-and-add 0 #acquire() 1996928
lock: kmem: #fetch-and-add 0 #acquire() 1879428
lock: kmem: #fetch-and-add 0 #acquire() 51
lock: kmem: #fetch-and-add 0 #acquire() 51
lock: kmem: #fetch-and-add 0 #acquire() 51
lock: kmem: #fetch-and-add 0 #acquire() 51
lock: kmem: #fetch-and-add 0 #acquire() 51
lock: bcache: #fetch-and-add 0 #acquire() 6175
lock: bcache: #fetch-and-add 0 #acquire() 6176
lock: bcache: #fetch-and-add 0 #acquire() 6322
lock: bcache: #fetch-and-add 0 #acquire() 6318
lock: bcache: #fetch-and-add 0 #acquire() 4270
lock: bcache: #fetch-and-add 0 #acquire() 4260
lock: bcache: #fetch-and-add 0 #acquire() 2625
lock: bcache: #fetch-and-add 0 #acquire() 4624
lock: bcache: #fetch-and-add 0 #acquire() 3792
lock: bcache: #fetch-and-add 0 #acquire() 4126
lock: bcache: #fetch-and-add 0 #acquire() 4172
lock: bcache: #fetch-and-add 0 #acquire() 6178
lock: bcache: #fetch-and-add 0 #acquire() 6174
--- top 5 contended locks:
lock: proc: #fetch-and-add 279665 #acquire() 2884996
lock: proc: #fetch-and-add 276589 #acquire() 2884997
lock: proc: #fetch-and-add 259698 #acquire() 2885026
lock: proc: #fetch-and-add 258801 #acquire() 2884996
lock: proc: #fetch-and-add 256363 #acquire() 2884996
tot= 0
test0: OK
start test1
test1 OK
```

Usertests:

```
test exitiput: OK
test iput: OK
test mem: OK
test pipe1: OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
```

Make grade:

```
== Test running kallocetest ==
$ make qemu-gdb
(59.0s)
== Test kallocetest: test1 ==
kallocetest: test1: OK
== Test kallocetest: test2 ==
kallocetest: test2: OK
== Test kallocetest: sbrkmuch ==
$ make qemu-gdb
kallocetest: sbrkmuch: OK (5.2s)
== Test running bcachetest ==
$ make qemu-gdb
(6.8s)
== Test bcachetest: test0 ==
bcachetest: test0: OK
== Test bcachetest: test1 ==
bcachetest: test1: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (82.0s)
== Test time ==
time: OK
Score: 70/70
```